

ACADEMIC PROJECT · CDE2313

TDM Insight

A Vancomycin Therapeutic Drug Monitoring app for Android

Architecture · Jetpack Compose UI · Kotlin concepts · Supabase backend · Clerk authentication

Kotlin

Jetpack Compose

Material 3

MVVM

Supabase

Clerk

Presented by

Mowlid Haibe · Abdinaazir Mustafe · Elham Ahmedngus

Supervisor: Ts. Mohd Zulkifli Mohd Zaki — Faculty of CS & IT

Albukhary International University (AIU)

What this presentation covers

01

The clinical problem

Why vancomycin needs individualised dosing

03

System architecture

The system diagram, the five layers, the dependency rule

05

Kotlin concepts & variables

The language features the app is built from

07

Backend & database

Supabase PostgreSQL, schema, persistence flow

02

The complete tech stack

Android, backend, auth and the landing page

04

The UI layer

Jetpack Compose, state, Material 3, navigation

06

The calculation engine

Formulas, workflow diagram, a worked example

08

Authentication & security

Clerk, OAuth, and credential management

Why vancomycin dosing cannot be one-size-fits-all

Vancomycin is a narrow-therapeutic-index antibiotic: too little fails the infection, too much damages the kidneys.

- Efficacy and toxicity both track the AUC_{24}/MIC ratio, not a single trough number.
- The 2020 Rybak guidelines set an AUC_{24} target of 400–600 mg·h/L.
- Every patient clears the drug differently — weight, age, sex and renal function all change the answer.
- Ward calculations are done by hand: slow, error-prone, and hard for students to learn from.

So the app must...

- Take patient data + measured drug concentrations
- Fit a one-compartment pharmacokinetic model
- Report AUC_{24} with a target verdict and a recommended dose
- Show every formula, so it teaches instead of just answering

AUC_{24} TARGET BAND

> 600 Above target — nephrotoxicity risk

400 – 600 In target — therapeutic

< 400 Below target — treatment failure

Rybak MJ et al., Am J Health-Syst Pharm, 2020

Educational tool — fictional patient data only.

What TDM Insight delivers

One offline-capable Android app that takes a case from raw patient data to an explained dosing recommendation.

3

sampling workflows

PRE · POST · PRE_POST

8

PK parameters

k_e , $t_{1/2}$, V_d , V_d/kg , CL , C_{min} , C_{max} , AUC_{24}

14

Compose screens

single-activity navigation graph

4

explanation phases

every formula shown with real numbers

1

Calculate

Cockcroft–Gault $CrCl$, Sawchuk–Zaske and Newton–Raphson fitting run on-device in pure Kotlin.

2

Explain

A four-phase walkthrough substitutes the user's own numbers into every equation.

3

Remember

Each completed case is written to Supabase and reloaded in the History screen.

4

Protect

Clerk handles sign-up, email verification and Google OAuth before any case can be created.

Every technology used, and why

Two independent projects: the Android application, and the React landing page that distributes it.

Android application

Kotlin 2.x (JVM 21)

Language — null-safe, concise, coroutine-native

Jetpack Compose

Declarative UI toolkit — no XML layouts at all

Material 3

Design system — colour scheme, typography, shapes

Navigation Compose

Single-activity routing between 14 screens

ViewModel + StateFlow

MVVM state holder, survives rotation

Coroutines

Async work off the main thread

Backend, auth & web

Supabase (PostgreSQL)

Hosted database + auto-generated REST API

Supabase Kotlin SDK

Typed Postgrest queries from the app

Clerk

Identity: email/password, verification, Google OAuth

Ktor + kotlinx.serialization

HTTP client & JSON for the Clerk Frontend API

React 19 + Vite 6 + TypeScript

Landing page — fast, typed, statically built

Gradle Kotlin DSL

Build system + version catalog + BuildConfig injection

Five layers, one direction of dependency

Each layer knows only the layer beneath it. The domain layer at the centre knows nothing about Android at all.

1

PRESENTATION

Jetpack Compose UI

SplashScreen · AuthScreens · HomeScreen · CaseWizardScreens · ResultScreens · HistoryScreen · SettingsScreens · theme/ · navigation/NavGraph

2

PRESENTATION LOGIC

ViewModel + StateFlow

CaseViewModel · AuthViewModel · HistoryViewModel — hold UI state, expose it as StateFlow, never touch Compose

3

DOMAIN

Pure Kotlin business logic

VancoEngine — Cockcroft–Gault, Sawchuk–Zaske, Newton–Raphson. No Android imports, unit-testable on the JVM.

4

DATA

Models, validation, repositories

PatientInput · DosingInput · PkResults · InputValidator · SupabaseRepository · AuthRepository

5

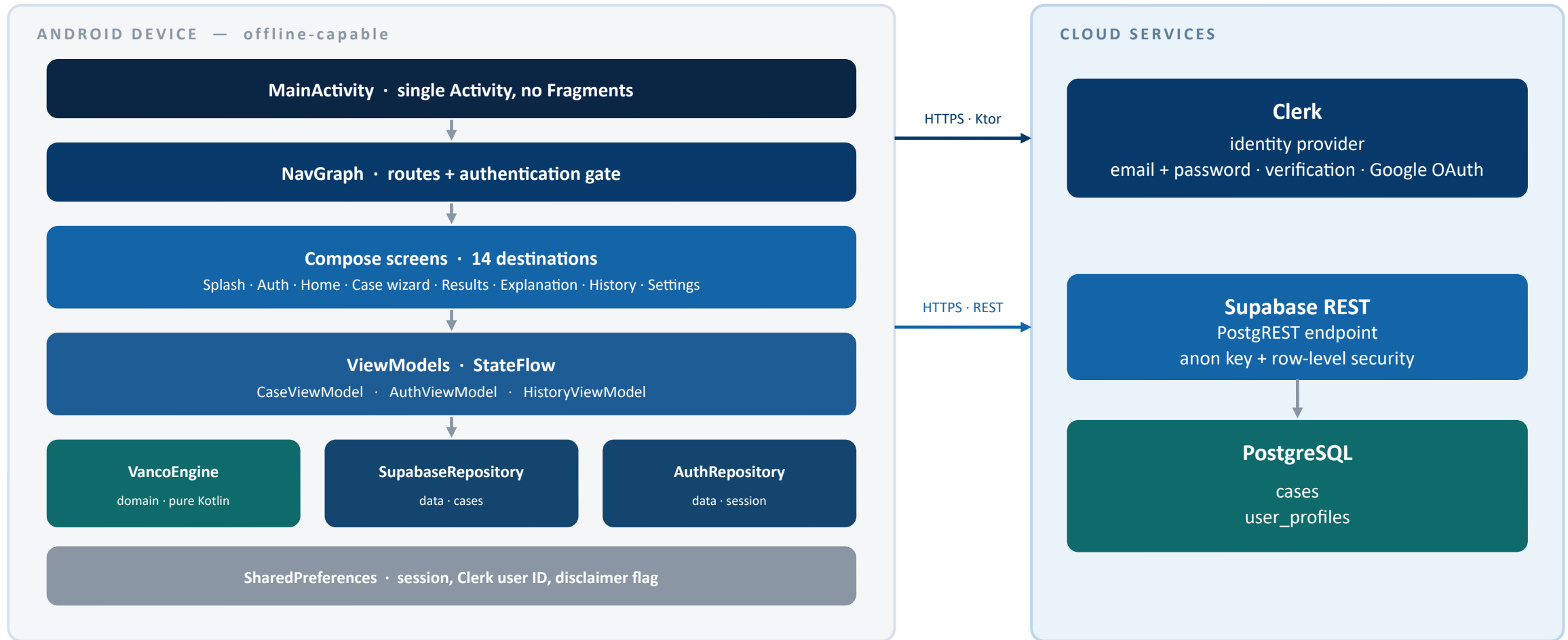
INFRASTRUCTURE

External services

Supabase PostgreSQL (REST) · Clerk Frontend API (Ktor/HTTP) · SharedPreferences (local session)

System architecture — the whole system on one page

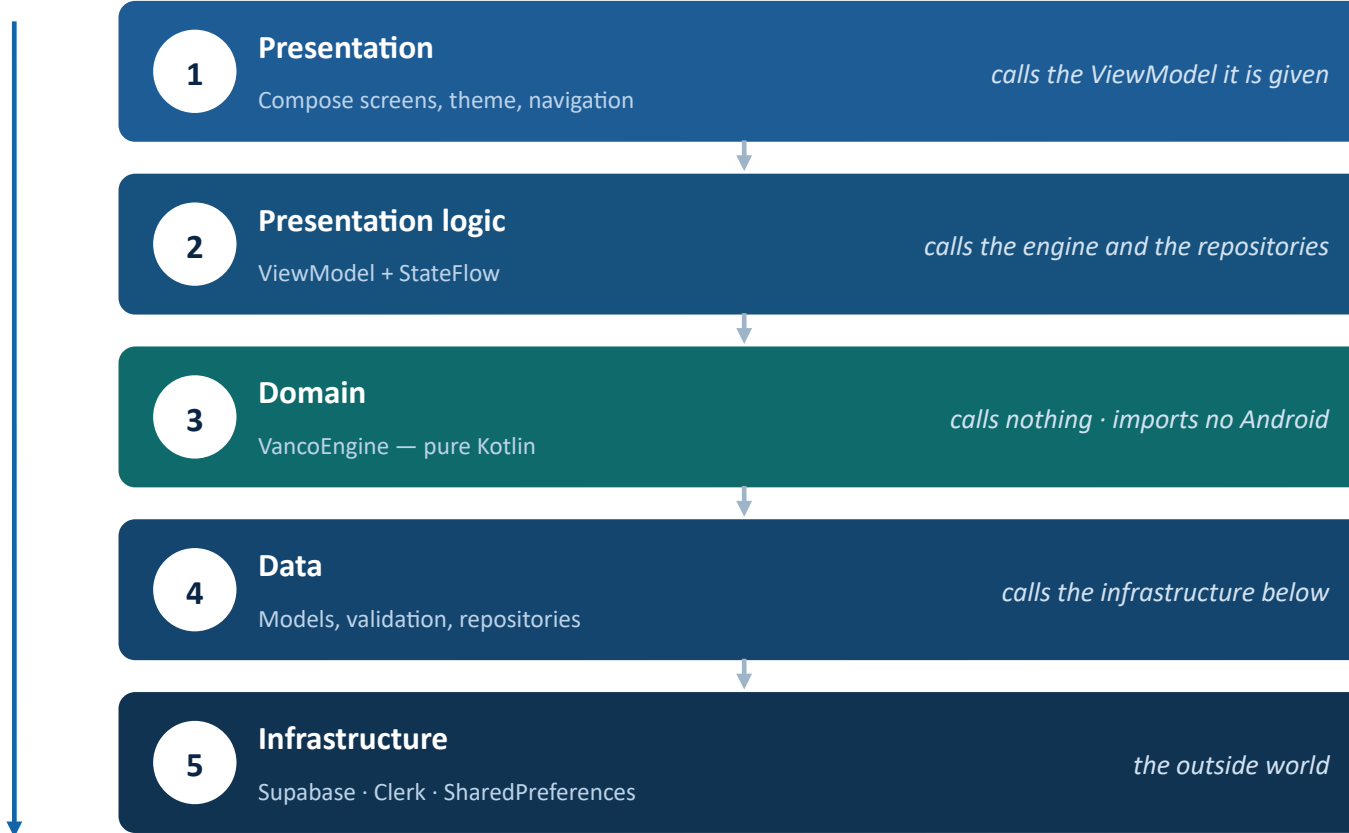
Everything on the left runs on the phone. Only two arrows cross into the cloud, and both are HTTPS.



The layer design — dependencies point one way

A layer may call the layer below it and nothing above it. Follow the arrow: it never turns back up.

DEPENDS ON



The rule

Dependencies point down only. No layer holds a reference to the layer above it, and no layer skips a level.

What it buys

The engine can be reviewed and unit-tested with no emulator. The UI, the database or the auth provider can each be swapped without touching the clinical maths.

The test

Delete the whole ui package and VancoEngine still compiles and its tests still pass. That is what proves the layering is real and not just folder names.

Layer 1 — Presentation, built entirely in Compose

There is not a single XML layout file in the project. Every pixel is a Kotlin function.

```
@Composable
fun ResultsScreen(
    navigationController: NavController,
    vm: CaseViewModel,
) {
    val state by vm.uiState.collectAsState() // observe

    Column(modifier = Modifier.padding(16.dp)) {
        AucRing(value = state.result?.auc24) // recompute
        PkParameterGrid(state.result)
    }
}
```

How to read this

- `@Composable` marks a function that emits UI — it describes what the screen should look like for the current state.
- `collectAsState()` subscribes the screen to the ViewModel's StateFlow.
- When state changes, Compose recomposes only the parts that read it.

Screen inventory

Splash & Disclaimer	entry gate
AuthScreens	sign in / sign up / verify code
HomeScreen	dashboard + recent cases
CaseWizardScreens	medication → workflow → inputs → review
CalculatingScreen	progress while the engine runs
ResultScreens	AUC ring, PK grid, explanation
HistoryScreen	cases reloaded from Supabase
Profile & Settings	account, data, about

Jetpack Compose — the concepts I applied

Compose is declarative: I never mutate a view, I describe the UI as a function of state and let the framework redraw.

1

Composable functions

UI is written as functions annotated `@Composable`. They take data in and emit UI — no `findViewById`, no view references.

2

Recomposition

When a value a composable reads changes, Compose re-runs only that function. This is why the AUC ring updates the instant a field changes.

3

State & remember

`mutableStateOf()` creates observable state; `remember` keeps it across recompositions; `rememberSaveable` survives rotation.

4

State hoisting

Child composables hold no state of their own — they receive value + `onValueChange`. State lives in the `ViewModel`, so it is testable and shareable.

5

Modifier chain

Layout, padding, size, click behaviour and styling are composed as an ordered Modifier chain instead of XML attributes.

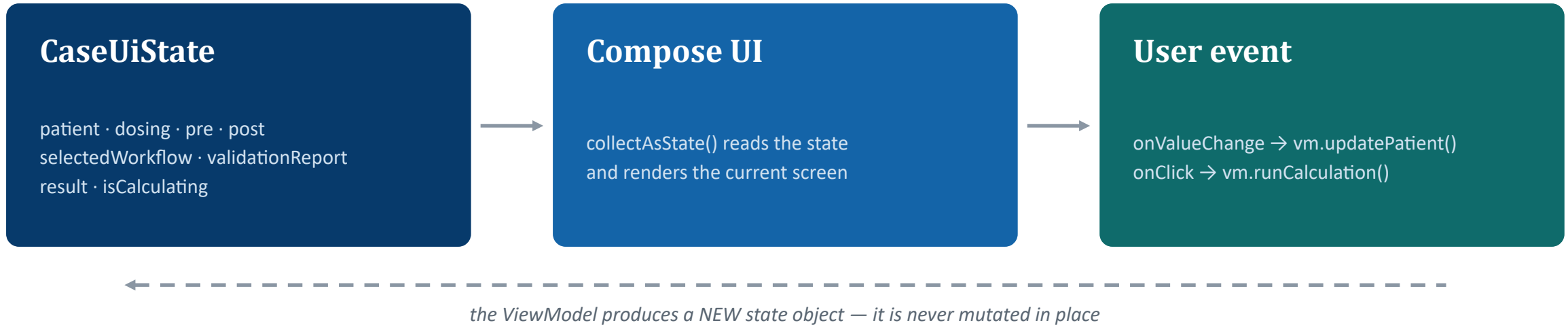
6

Layouts & lists

`Column`, `Row` and `Box` replace `LinearLayout/RelativeLayout`; `LazyColumn` replaces `RecyclerView` with no adapter or `ViewHolder`.

Unidirectional data flow — state down, events up

One immutable state object describes the whole wizard. The UI never changes it directly; it only sends events.



```
private val _uiState = MutableStateFlow(CaseUiState())
val uiState: StateFlow<CaseUiState> = _uiState.asStateFlow()

fun updatePatient(patient: PatientInput) {
    _uiState.update { it.copy(patient = patient) } // immutable
}
```

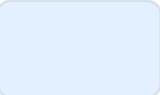
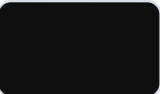
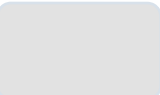
Why this matters

- Only one source of truth — no desynchronised widgets
- Survives screen rotation with the ViewModel
- State is a data class, so it is trivial to test

Material 3 — one design system, applied everywhere

Colour, typography and shape are defined once in `ui/theme/` and inherited by every screen.

Colour scheme — `ui/theme/Color.kt`

	Primary	#1464A8 — clinical blue
	PrimaryContainer	#E3FOFF — tinted surfaces
	OnSurface	#0F0F0F — near-black text
	OnSurfaceVariant	#6B6B6B — secondary text
	Outline	#E2E2E2 — card borders
	Error	#CC2929 — validation errors

Typography

Type.kt defines the Material 3 type scale — `displayLarge` down to `labelSmall`. Results use monospace-style figures so digits line up in the PK grid.

Shape

Shape.kt sets the corner radii used by every card, chip and button, so the whole app shares one geometry.

Light & dark

Theme.kt builds a `lightColorScheme()` and a `darkColorScheme()` from the same tokens. Switching theme changes no screen code.

Components

Scaffold, TopAppBar, Card, Button, OutlinedTextField, LinearProgressIndicator — all Material 3, all reading the theme.

Navigation — one Activity, a graph of routes

MainActivity hosts a single NavHost. Every screen is a destination identified by a String route.

```
object Routes {  
    const val HOME          = "home"  
    const val WORKFLOW_SELECT = "workflow_select"  
    const val INPUT_FORM    = "input_form"    // {workflow} arg  
    const val RESULTS        = "results"  
    const val HISTORY        = "history"  
}  
  
composable("${Routes.INPUT_FORM}/{workflow}") { entry ->  
    InputFormScreen(nav, entry.arguments?.getString("workflow"))  
}
```

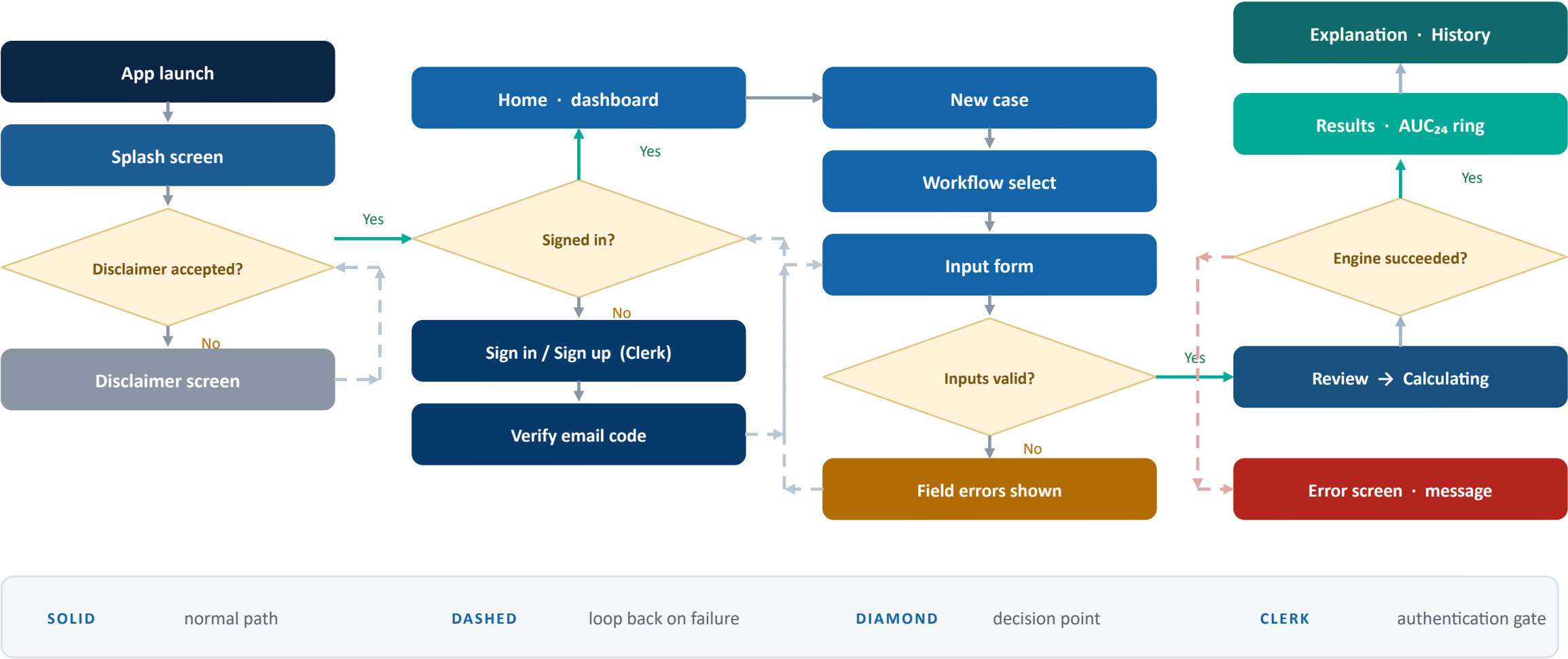
- Single-activity architecture — no Fragments, no Intents between screens.
- The workflow (PRE / POST / PRE_POST) is passed as a navigation argument.
- An auth gate in NavGraph sends unauthenticated users to the sign-in route before the graph is even reachable.

The case flow

-
- Splash
 - Disclaimer
 - Sign in (Clerk)
 - Home
 - New case
 - Workflow select
 - Input form
 - Review
 - **Calculating**
 - **Results**
 - **Explanation**

Screen flow — the path a user takes

Two gates protect the app: the disclaimer, and the authentication check. The dashed arrows are the failure loops.



Layer 2 — MVVM with ViewModel and StateFlow

Three ViewModels own all screen state. They outlive configuration changes and never import anything from Compose.

CaseViewModel

Owns CaseUiState: patient, dosing, samples, workflow, validation and result. Calls VancoEngine, then hands the result to SupabaseRepository.

AuthViewModel

Exposes AuthState (Loading / Unauthenticated / AwaitingEmailCode / Authenticated / Error) from AuthRepository and Clerk.

HistoryViewModel

Loads the 20 most recent cases for the signed-in user from Supabase and exposes them to the History and Home screens.

```
fun runCalculation() {
    validate()
    val s = _uiState.value
    if (!s.validationReport.isValid) return

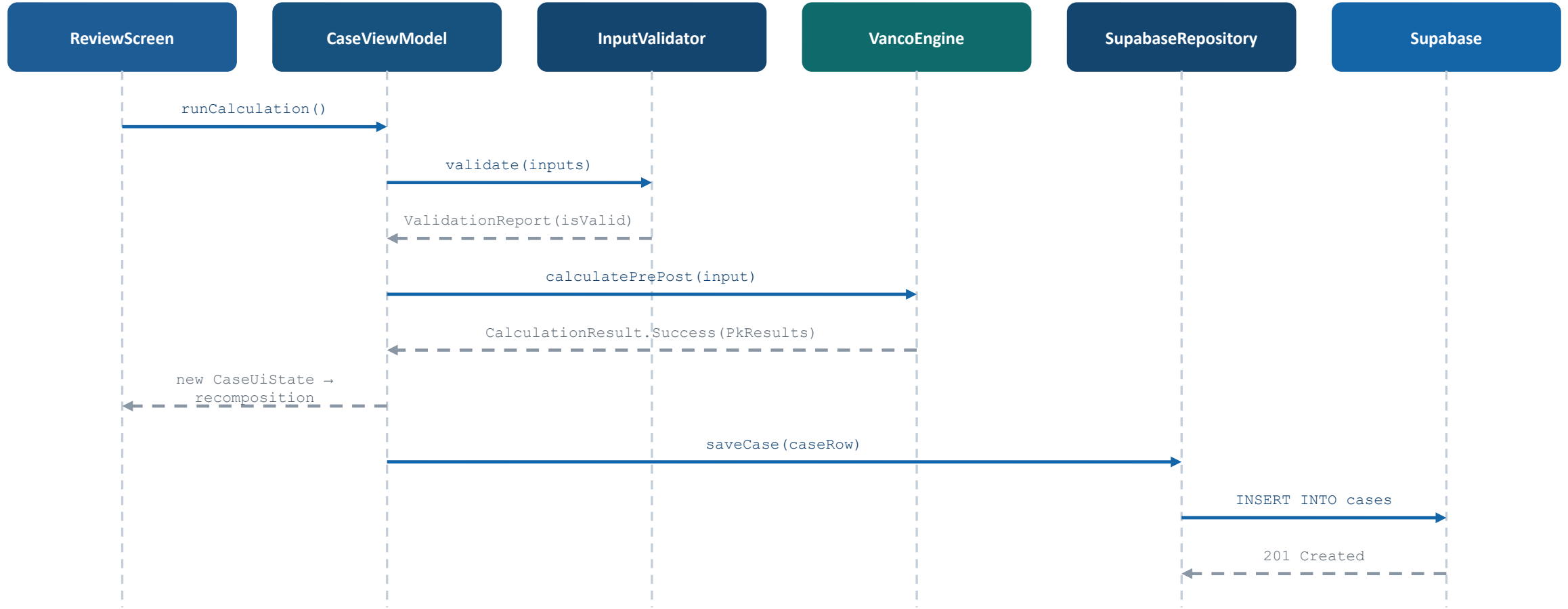
    viewModelScope.launch { // coroutine
        val result = withContext(Dispatchers.Default) {
            VancoEngine.calculatePrePost(...) // CPU work
        }
        _uiState.update { it.copy(result = result) }
        launch { supabaseRepository.saveCase(...) } // I/O
    }
}
```

Three things to notice

- viewModelScope cancels the coroutine automatically when the screen dies — no leaks.
- Dispatchers.Default moves the pharmacokinetic maths off the main thread, so the UI never freezes.
- The Supabase write is fire-and-forget: a network failure can never hide a valid result from the user.

Sequence diagram — running one calculation

Solid arrows are calls, dashed arrows are returns. Notice the database write happens after the user already has the answer.



The engine runs on Dispatchers.Default, the insert on Dispatchers.IO, both off the main thread. If the report is invalid the sequence stops at step 3 — the engine is never called.

The Kotlin features this project is built from

Each concept below is used deliberately — these are the constructs I would point to in a code walkthrough.

<code>data class</code>	PatientInput, DosingInput, PkResults, CaseUiState	Free equals/hashCode/toString and copy() — the basis of immutable state updates.
<code>sealed class</code>	CalculationResult (Success Failure), AuthState	A closed set of outcomes the when block must handle exhaustively — no forgotten branch.
<code>enum class</code>	VancoWorkflow (PRE, POST, PRE_POST), Auc24Verdict	A fixed vocabulary the compiler enforces instead of magic strings.
<code>object (singleton)</code>	VancoEngine, SupabaseClientProvider, Routes	One instance for the whole process — the engine is stateless, so it needs no more.
<code>Null safety</code>	Double? for optional PK parameters; ?. and ?: everywhere	Not every workflow produces every parameter, and the type system says so.
<code>Coroutines & Flow</code>	viewModelScope.launch, suspend fun, StateFlow	Structured concurrency: async work is scoped, cancellable and off the main thread.
<code>Higher-order & lambdas</code>	_uiState.update { it.copy(...) }, onValueChange = { }	Behaviour passed as a value — how Compose callbacks and state updates are written.
<code>Immutability (val)</code>	Almost every declaration in the project is val	State is replaced, not mutated — which is exactly what recomposition needs.

Variables and data types — the real ones in the code

Every clinical quantity is a typed val with its unit written into the property name.

```
data class PatientInput(
    val caseId: String    = "",        // case label
    val weightKg: Double = 0.0,        // kg
    val heightCm: Double = 0.0,        // cm
    val ageLYears: Int    = 0,          // years
    val isMale: Boolean   = true,       // sex factor
    val scrUmolL: Double = 0.0,        // umol/L
)

val kePerHour: Double? = null    // nullable: not every
val auc24: Double?     = null    // workflow yields it
```

val vs var

- val = read-only reference (used almost everywhere) — a new object is created instead of editing the old one.
- var appears only inside the Newton–Raphson loop, where a value genuinely has to change between iterations.

Types used, and why

Double	All clinical measurements — concentrations, doses, times, PK parameters. Precision matters.
Int	Age in whole years.
Boolean	isMale (Cockcroft–Gault sex factor), isCalculating.
String	caseId, Clerk userId, route names, error messages.
Double?	Optional PK outputs — null means “this workflow does not produce it”.
List<HistoryEntry>	The 20 most recent cases returned by Supabase.
enum / sealed	Workflow selection and calculation outcome.

Layer 3 — VancoEngine, pure Kotlin with zero Android

The whole clinical model lives in one stateless object. It imports kotlin.math and nothing else from the platform.

```
object VancoEngine {
    private const val AUC24_TARGET_LOW      = 400.0
    private const val AUC24_TARGET_HIGH    = 600.0
    private const val AUC24_TARGET_CENTRE  = 500.0
    private const val POP_VD_L_PER_KG      = 0.7
    private const val CRCL_TO_CL_FACTOR    = 0.06

    fun calculatePre(input: PreWorkflowInput): CalculationResult
    fun calculatePost(input: PostWorkflowInput): CalculationResult
    fun calculatePrePost(i: PrePostWorkflowInput): CalculationResult
}
```

Why the domain layer is separate

- The clinical maths can be reviewed by a pharmacist without reading any UI code.
- The UI, the database and the auth provider could all be replaced without touching a single formula.

Stateless

An object, not a class — no fields, no lifecycle, safe to call from any thread.

Testable

No Android imports means it runs in a plain JVM unit test, no emulator required.

Total

Every entry point returns CalculationResult — Success or Failure. It never throws at the caller.

Guarded

guardPositive() rejects a zero or negative clearance, interval or ke before it can produce a nonsense answer.

Sourced

Every constant is annotated with its clinical reference in the source comments.

The calculation, step by step

Five equations turn patient data and a measured concentration into a dosing recommendation.

1	Creatinine clearance	$\text{CrCl} = (140 - \text{age}) \times \text{weight} \times \text{sexFactor} / (72 \times \text{SCr})$	Cockcroft–Gault. SCr converted $\mu\text{mol/L} \rightarrow \text{mg/dL}$ by $\div 88.4$; sexFactor = 1.0 male, 0.85 female.
2	Clearance & volume	$\text{CL} = \text{CrCl} \times 0.06$ $\text{Vd} = 0.7 \text{ L/kg} \times \text{weight}$	CrCl in mL/min becomes vancomycin clearance in L/h. Vd uses the population prior.
3	Elimination constant	$\text{ke} = \text{CL} / \text{Vd}$ $t_{1/2} = \ln(2) / \text{ke}$	How fast the drug leaves. Half-life follows directly from ke.
4	Exposure	$\text{AUC}_{24} = (\text{Dose} / \tau) \times 24 / \text{CL}$	The primary TDM target — total 24-hour exposure in mg·h/L.
5	Recommendation	$\text{Dose} = \text{AUC_target} \times \text{CL} \times \tau / 24$	Rearranged step 4, solved for the dose that centres AUC_{24} on 500 mg·h/L.

TDM Insight, CDE2313, Albury International University (AIU)
 τ = dosing interval in hours. All five steps run in `VancoEngine.calculatePre()`; the other two workflows replace steps 2–3 with a fitted ke.

A worked example, end to end

Male, 65 years, 70 kg, serum creatinine 90 $\mu\text{mol/L}$, receiving 1000 mg every 12 h over a 1 h infusion.

SCr 90 $\mu\text{mol/L}$ $\rightarrow$ 1.018 mg/dL

CrCl $(140 - 65) \times 70 \times 1.0 / (72 \times 1.018) = 71.6 \text{ mL/min}$

CL $71.6 \times 0.06 = 4.30 \text{ L/h}$

Vd $0.7 \times 70 = 49.0 \text{ L}$

ke $4.30 / 49.0 = 0.0877 \text{ h}^{-1}$

t_{1/2} $\ln(2) / 0.0877 = 7.9 \text{ h}$

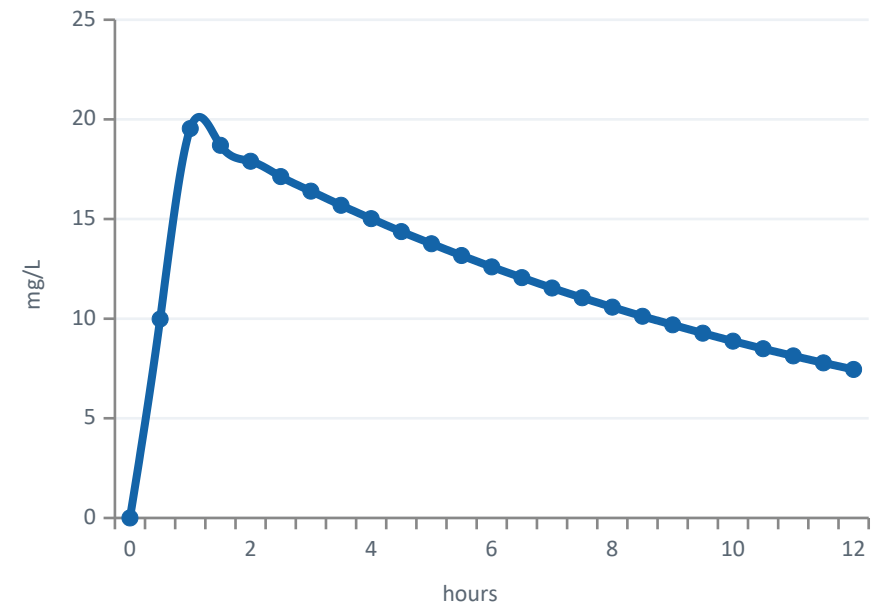
AUC₂₄ $(1000 / 12) \times 24 / 4.30 = 465 \text{ mg} \cdot \text{h/L}$

Dose $500 \times 4.30 \times 12 / 24 = 1074 \text{ mg every 12 h}$

AUC₂₄ 465 mg·h/L $\rightarrow$ IN TARGET (400–600)

Predicted concentration–time curve

One-compartment model, hours after dose start



Peak $\approx 19.5 \text{ mg/L}$ at end of infusion · trough $\approx 7.5 \text{ mg/L}$ at 12 h

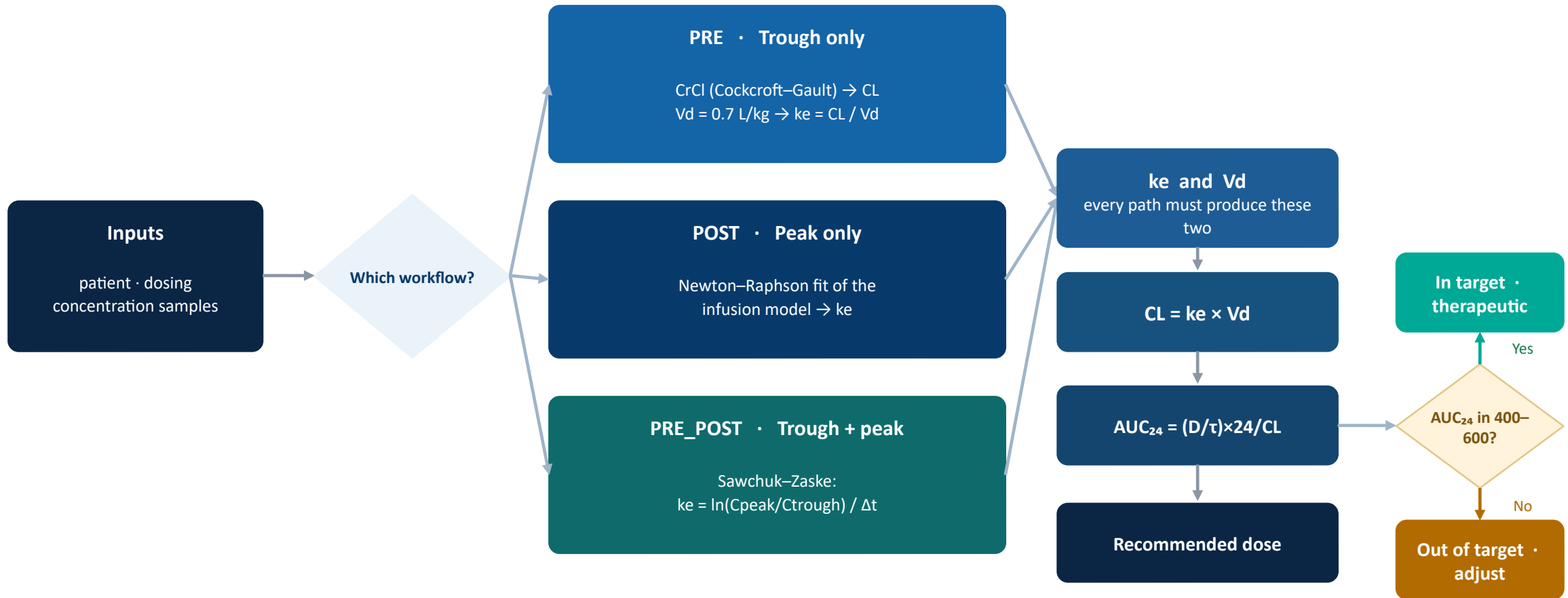
Three sampling workflows, one result type

The workflow the user picks decides how k_e is obtained. Everything downstream of k_e is identical.

PRE	POST	PRE_POST
Trough only	Peak only	Trough + peak
Sample Pre-dose trough ke from Population Vd + CrCl Method Cockcroft–Gault Use when Only a trough level is available Trade-off Fastest, but relies on population priors	Sample One post-dose level ke from Iterative curve fit Method Newton–Raphson on the infusion model Use when Only a post-dose level was drawn Trade-off Fits k_e from real data, still assumes Vd	Sample Both trough and peak ke from Two measured points Method Sawchuk–Zaske log-linear regression Use when Both levels are available Trade-off Most patient-specific — the default

The calculation workflow — three paths, one answer

The workflows differ in exactly one place: how k_e is obtained. After that they converge completely.



Recommended dose = $AUC_{target} \times CL \times \tau / 24$, with $AUC_{target} = 500 \text{ mg} \cdot \text{h/L}$ — the centre of the therapeutic band.

Validation and error handling

A clinical calculator must fail loudly, never silently. Nothing reaches the engine unchecked.

```
sealed class CalculationResult {
    data class Success(
        val workflow: VancoWorkflow,
        val intermediate: PkResults,
    ) : CalculationResult()

    data class Failure(val message: String) : CalculationResult()
}
```

The result is a closed set, so the UI must handle both branches — the compiler will not let a failure be ignored.

Five independent checks before a number is ever shown

Validation runs in the ViewModel before the engine is called, and again inside the engine itself. A user can never reach the Results screen with a value the model cannot support.

- 1 Field level**
 InputValidator checks each value as it is typed — weight, age, creatinine, dose, interval, sample times.
- 2 Cross-field**
 validateTimingRelation() confirms the trough is drawn after the peak, so Δt is positive.
- 3 Mathematical**
 guardLogSafety() blocks a log of zero or a negative ratio; guardPositive() rejects a non-positive CL, τ , ke or Vd.
- 4 Runtime**
 ArithmeticException is caught inside the engine and converted into a CalculationResult.Failure with a readable message.
- 5 UI**
 ErrorScreen shows the message; the Results screen is never reached with invalid numbers.

Layer 4 — Supabase as the database API

There is no server of my own to deploy. Supabase hosts the database and generates the REST API the app calls.



```
suspend fun saveCase(...): Boolean = withContext(Dispatchers.IO) {  
    client.from("cases").insert(caseRow)  
}  
  
internal suspend fun loadRecentCases(limit: Int = 20) =  
    client.from("cases")  
        .select()  
        .decodeList<CaseRow>()  
        .filter { it.userId == currentUserId }
```

What the repository guarantees

- Every network call is a suspend function on Dispatchers.IO — never on the main thread.
- Rows are stamped with the signed-in Clerk user ID, so History only ever shows your own cases.
- A failed write is logged and swallowed — the calculation the user is looking at is never lost.

Repository operations: saveCase() · loadRecentCases() · syncUserProfile() · loadUserProfile() · deleteAllCases() · deleteAllUserData()

The database schema

Two tables, created by supabase/schema.sql. One row in cases is one complete, auditable calculation.

cases

one row per completed calculation

Identity	id uuid PK · user_id text · created_at timestamptz
Case	case_label text · workflow text (PRE POST PRE_POST)
Patient	weight_kg · age_years · is_male · scr_umol_l
Regimen	dose_mg · interval_hours · infusion_duration_hours
Samples	pre_conc_mg_l · pre_time_h · post_conc_mg_l · post_time_h
PK results	ke_per_hour · half_life_hours · vd_l · vd_l_per_kg · clearance_l_per_hour
Outputs	auc24 · recommended_dose_mg · c_min · c_max

index: cases_user_created_idx (user_id, created_at desc)

user_profiles

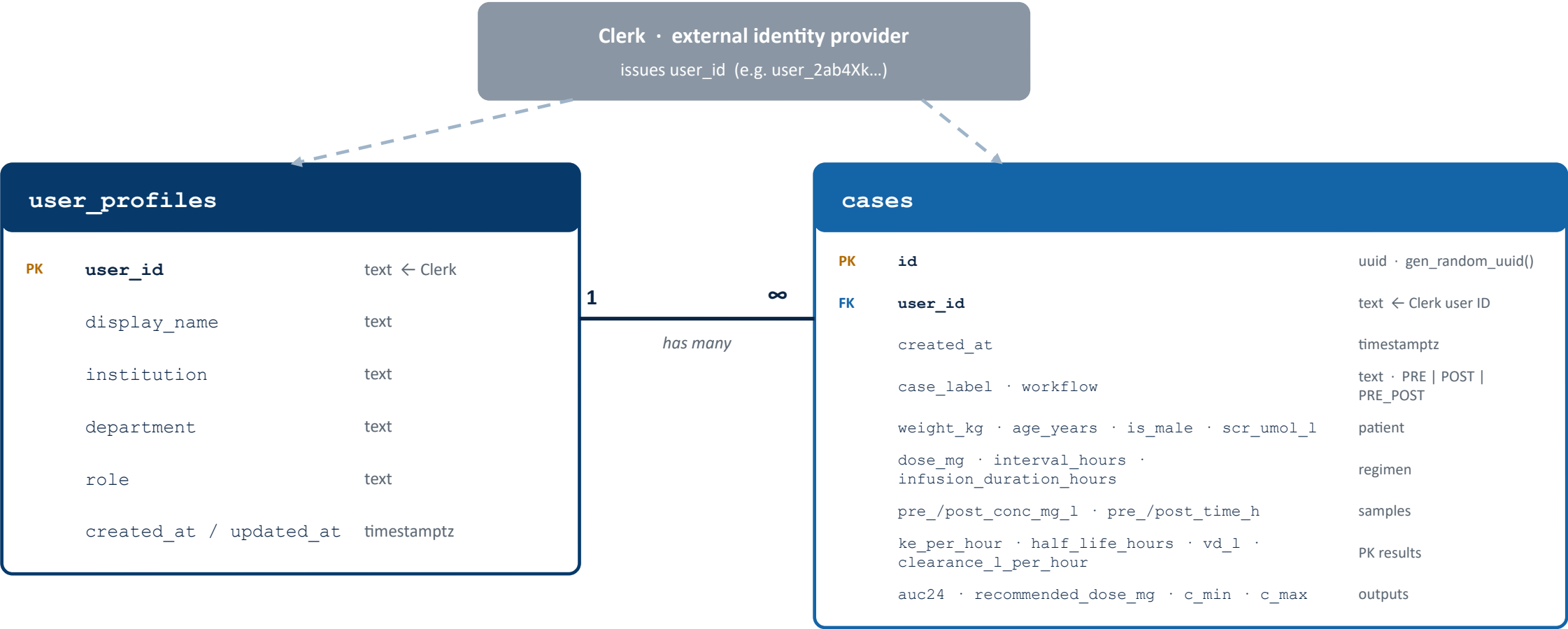
user_id text PK — the Clerk user ID
display_name · institution · department
role (student | pharmacist | doctor)
created_at · updated_at

Row-level security

RLS is enabled on both tables. For this academic build the anon policies allow read and insert, and the app filters by Clerk user ID. A production build would tighten these policies to the authenticated JWT.

Entity relationship diagram

Identity comes from Clerk, not from my database — the Clerk user ID is the key that joins the two tables.



Index: cases_user_created_idx (user_id, created_at desc) — serves the “20 most recent cases for this user” query. RLS enabled on both tables.

Clerk — identity without writing an auth server

The app calls Clerk's Frontend API directly over Ktor HTTP. No password is ever stored or hashed by my code.

1 Sign up

Email + password, or Continue with Google

2 Verify

Clerk emails a 6-digit code — `AuthState.AwaitingEmailCode`

3 Session

Clerk returns a session token, saved in `SharedPreferences`

4 Gate

`NavGraph` only exposes the app once `AuthState` is `Authenticated`

5 Identify

The Clerk user ID stamps every row written to Supabase

`AuthState` — a sealed class

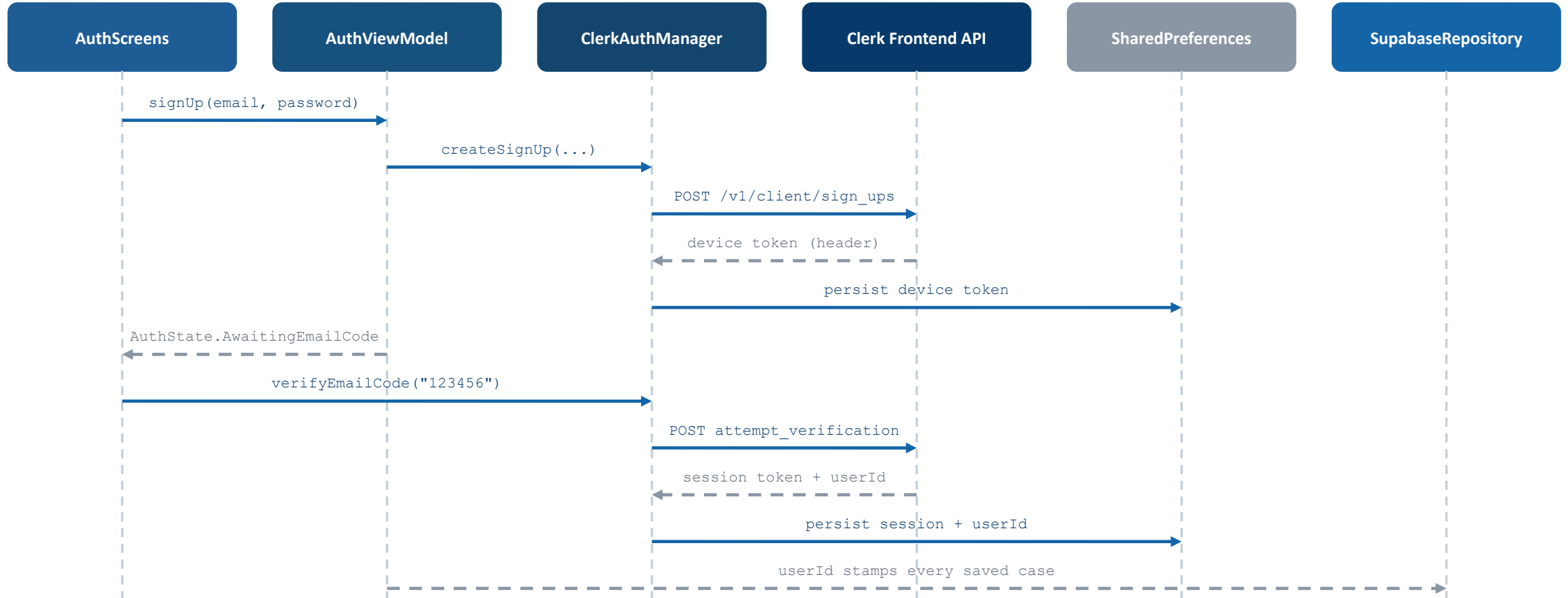
```
Loading
Unauthenticated
AwaitingEmailCode(email, busy)
Authenticated(userId, email, token, name...)
Error(message)
```

The native-client detail

Clerk's Frontend API normally tracks the device with a browser cookie. An Android app has no cookie jar, so every request sends `_is_native=1` and Clerk returns a device token in the `Authorization` header instead. That token is persisted and replayed, which is what lets the Google OAuth browser round-trip return to the same Clerk client via the deep link `tdminsight://oauth-callback`.

Authentication sequence — sign-up to signed-in

No password is ever stored by the app. Clerk owns the credential; the app only holds the returned session token.



Google OAuth follows the same shape: the device token is what lets the browser round-trip return to the same Clerk client via `tdminsight://oauth-callback`.

Credential management — no secret in the source tree

Keys are injected at build time from a gitignored file into BuildConfig, then read once at application start.

```
// app/build.gradle.kts - read the gitignored file
val localProps = Properties().also { p ->
    rootProject.file("local.properties")
        .inputStream().use { p.load(it) }
}
val clerkKey = localProps.getProperty("CLERK_PUBLISHABLE_KEY")
buildConfigField("String", "CLERK_PUBLISHABLE_KEY", clerkKey)

// TdmApplication.kt - read once, at app start
val clerkAuthManager by lazy {
    ClerkAuthManager(BuildConfig.CLERK_PUBLISHABLE_KEY, authPrefs)
}
```

local.properties is gitignored; secrets.defaults.properties is committed with placeholders so the structure is visible without exposing a key.

The principle

Anything shipped inside an APK can be extracted. So only keys that are safe when public go in — everything else stays on the provider's servers.

Safe in a client app

- Clerk publishable key (pk_test_...) — public by design
- Supabase project URL — not a secret
- Supabase anon key — restricted by row-level security

Never in the APK

- Clerk secret key (sk_test_...) — server-side only
- Supabase service_role key — bypasses RLS entirely
- Clerk JWKS private key — server-side only

The landing page — how the app is distributed

A separate React project that presents the app and serves the APK download.

React 19

Component-based UI, the same declarative idea as Compose but for the web

TypeScript 5.9

Static types across the whole site — the same safety argument as Kotlin

Vite 6

Dev server with hot reload; builds a static bundle for production

Tailwind 4

Utility-first styling alongside hand-written CSS for the animated sections

React Router 7

Client-side routing

Lucide React

Icon set used across the feature cards

What the page contains

- Hero with an interactive phone mock-up
- Animated screen carousel of all 7 app screens
- Feature grid — the four core capabilities
- Three-step “how it works” walkthrough
- Results explainer with the AUC ring
- Reviews marquee
- Team section — supervisor and developers
- Download call-to-action (APK + GitHub)

How it is built and shipped

```
npm run dev → local Vite dev server with hot reload
npm run build → type-check, then a static bundle in dist/
The release APK sits in public/TDM-Insight.apk and is served by the Download button.
```

One tap on Calculate — what actually happens

This is the single slide that connects the UI, the ViewModel, the engine and the backend.

01

UI event

ReviewScreen calls `vm.runCalculation()`. The composable itself does no work.

02

Validate

InputValidator re-checks patient, dosing, samples and their timing relation. A failure stops here.

03

Snapshot

The ViewModel reads one immutable `CaseUiState`, so the inputs and the verdict describe the same moment.

04

Compute

`viewModelScope.launch + Dispatchers.Default` runs `VancoEngine` off the main thread — the UI stays responsive.

05

Emit

`_uiState.update { it.copy(result = ...) }` produces a new state; Compose recomposes the Results screen.

06

Persist

A second coroutine writes the case to Supabase, stamped with the Clerk user ID. Failure is logged, never surfaced.

07

Recall

HistoryViewModel reads the 20 most recent rows back, and the case appears in History on the next visit.

What this project demonstrates

Architecture

A five-layer separation where the clinical logic has no dependency on Android, the UI or the network.

Modern Android

A single-activity, fully declarative Compose app with Material 3 theming and MVVM state management.

Kotlin depth

Data classes, sealed classes, objects, null safety, coroutines and immutable state used for concrete reasons.

Real integration

A live PostgreSQL backend and a production identity provider, wired safely through build-time configuration.

Thank you — questions welcome

Mowlid Haibe · Abdinaazir Mustafe · Elham Ahmedngus | Supervisor: Ts. Mohd Zulkifli Mohd Zaki

Albukhary International University (AIU) · Faculty of Computer Science & Information Technology · CDE2313